necessary to ensure that you don't get a data race reading the data from the head. If you didn't have that mutex, it would be quite possible for a thread to call `try_pop()` and a thread to call `push()` concurrently, and there'd be no defined ordering on their operations. Even though each member function holds a lock on a mutex, they hold locks on *different* mutexes, and they potentially access the same data; all data in the queue originates from a call to `push()`, after all. Because the threads would be potentially accessing the same data without a defined ordering, this would be a data race, as you saw in chapter 5, and undefined behavior. Thankfully the lock on the `tail_mutex` in `get_tail()` solves everything. Because the call to `get_tail()` locks the same mutex as the call to `push()`, there's a defined order between the two calls. Either the call to `get_tail()` occurs before the call to `push()`, in which case it sees the old value of `tail`, or it occurs after the call to `push()`, in which case it sees the new value of `tail` *and the new data attached to the previous value of* `tail`.

It's also important that the call to `get_tail()` occurs inside the lock on `head_mutex`. If it didn't, the call to `pop_head()` could be stuck in between the call to `get_tail()` and the lock on the `head_mutex`, because other threads called `try_pop()` (and thus `pop_head()`) and acquired the lock first, thus preventing your initial thread from making progress:

```
std::unique_ptr<node> pop_head()                    <- This is a broken implementation
{
    node* const old_tail=get_tail();               <- (1) Get old tail value outside lock on head_mutex
    std::lock_guard<std::mutex> head_lock(head_mutex);

    if(head.get()==old_tail)                        <- (2)
    {
        return nullptr;
    }
    std::unique_ptr<node> old_head=std::move(head);
    head=std::move(old_head->next);                 <- (3)
    return old_head;
}
```

In this *broken* scenario, where the call to `get_tail(0)` (1) is made outside the scope of the lock, you might find that both `head` and `tail`